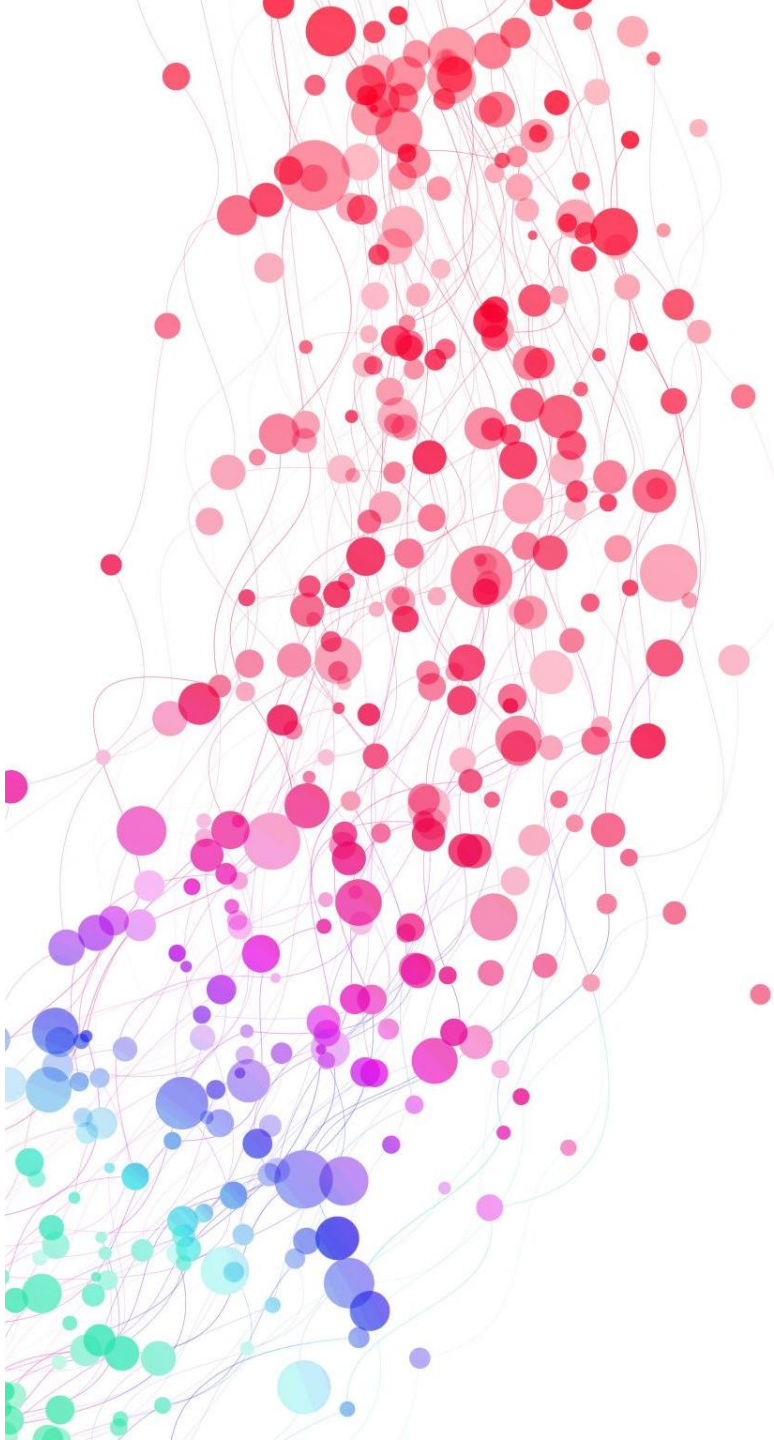
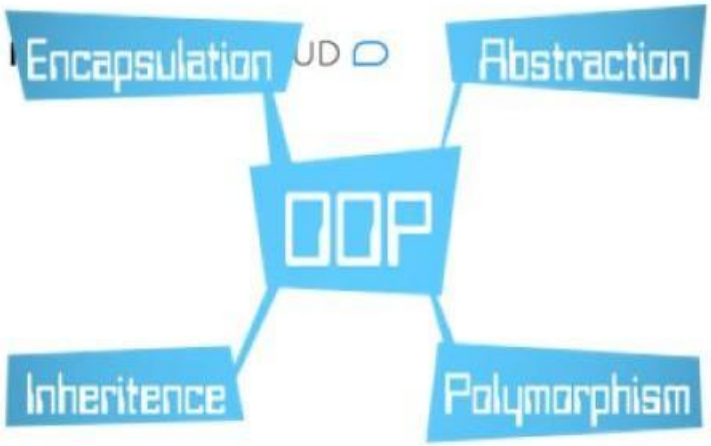
LECTURE 2

Computer programming II



REMEMBER



OOP Principles

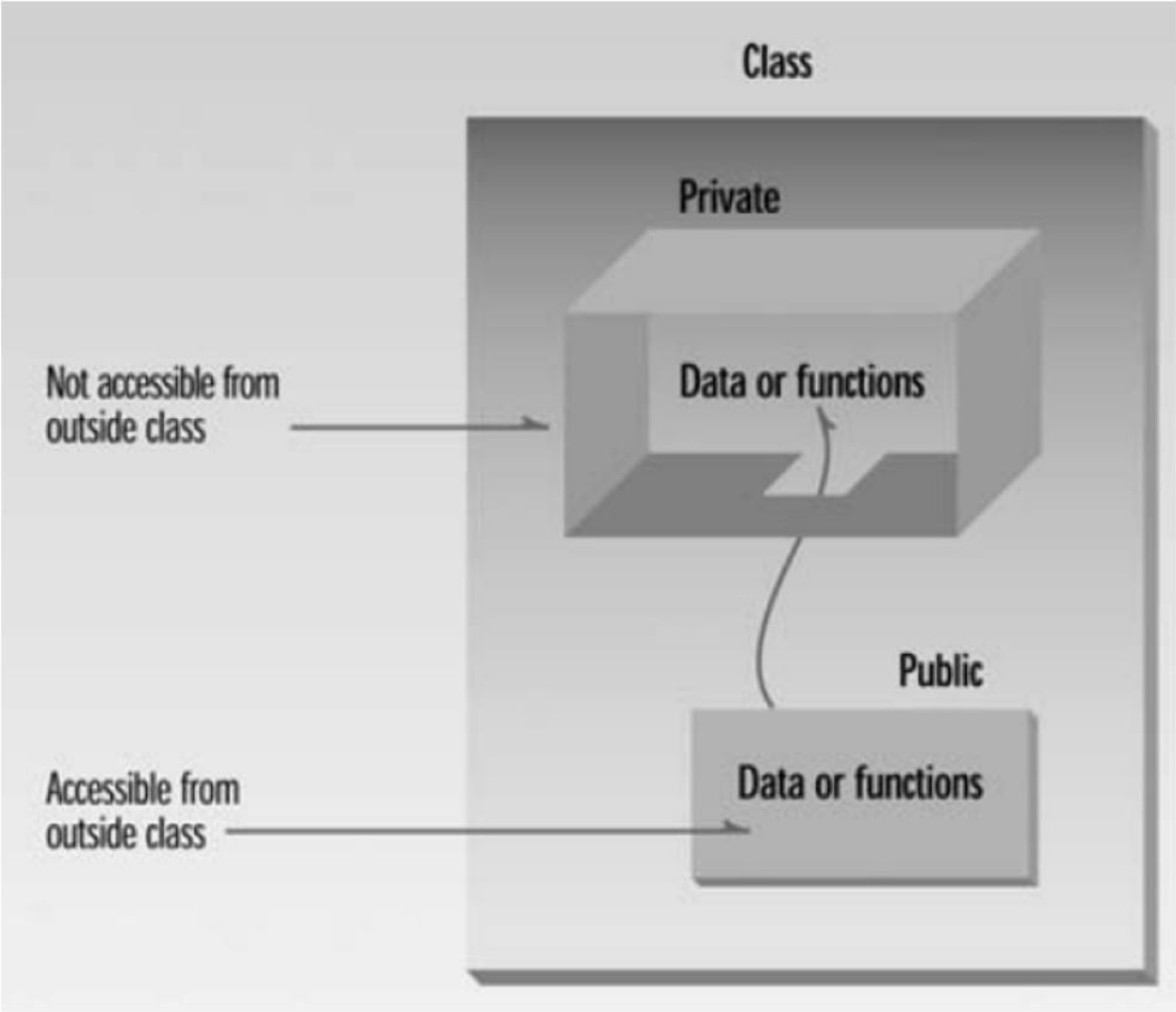
Encapsulation	Abstraction	Inheritance	Polymorphism
When an object only exposes the selected information.	Hides complex details to reduce complexity.	Entities can inherit attributes from other entities.	Entities can have more than one form.

Abstraction

```cpp
#include <iostream>
using namespace std;

class Adder {
public:
   // interface to outside world
   void addNum(int number) {
      total += number;
   }

   // interface to outside world
   int getTotal() {
      return total;
   };

private:
   // hidden data from outside world
   int total;
};
```

Abstraction refers to providing only essential information about the data to the outside world, ignoring unnecessary details or implementation.

```cpp
int main() {
   Adder a;

   a.addNum(10);
   a.addNum(20);
   a.addNum(30);

   cout << "Total " << a.getTotal() <<endl;
   return 0;
}
```


Class
Private
Data or functions
Not accessible from
outside class
Public
Data or functions
Accessible from
outside class

Keyword

Name of class

```
class foo
{
private:
   int data;
public:
   void memfunc (int d)
      { data = d; }
};
```

Braces

Keyword private and colon

Private functions and data

Keyword public and colon

Public functions and data

Semicolon

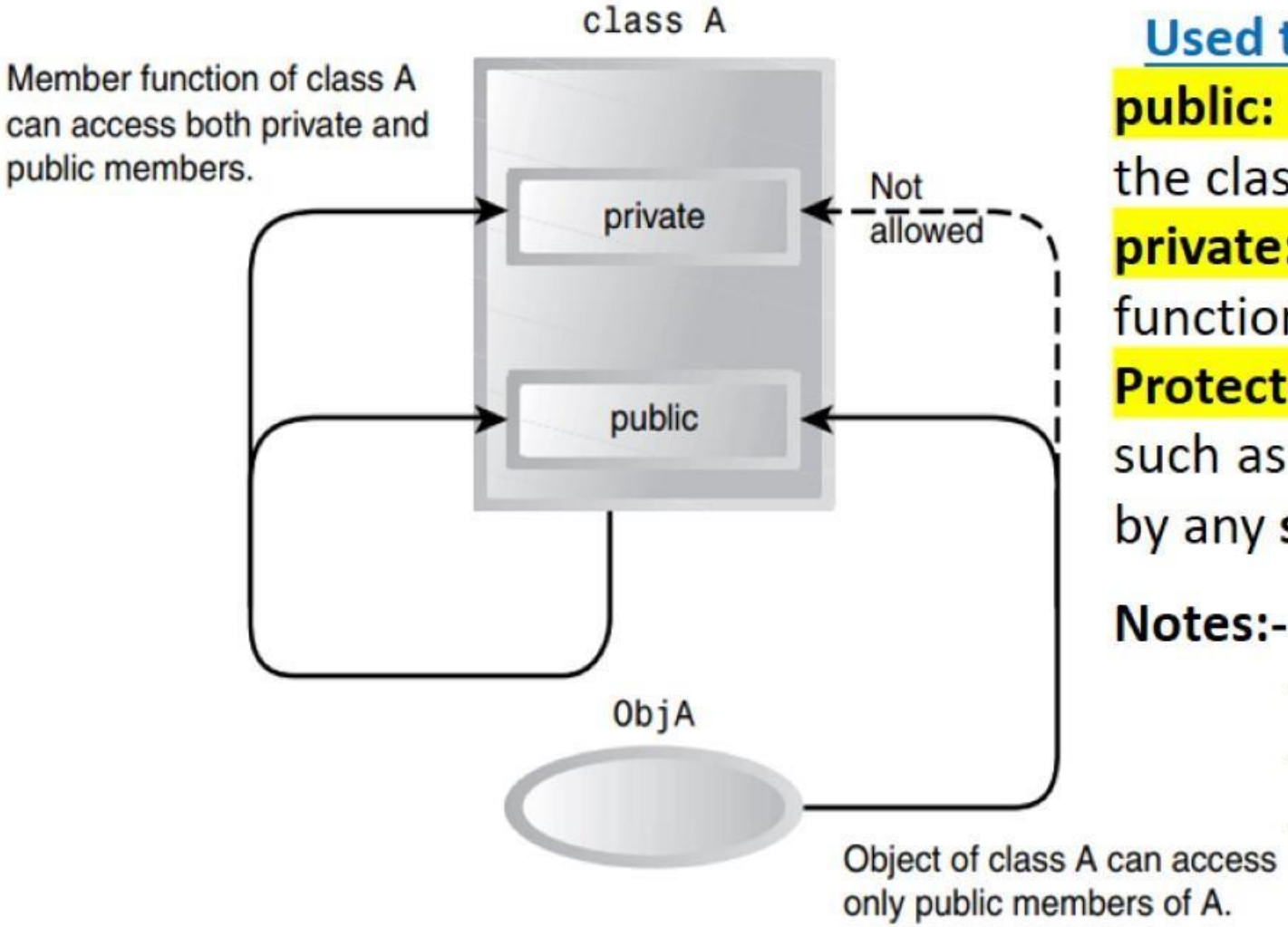
Circle
- radius: double
+ getArea(): double + getCircumference(): double

Instances / objects

c1:Circle
- radius=0
+ getArea(): double + getCircumference(): double

c2:Circle
- radius=5
+ getArea(): double + getCircumference(): double

Access Modifiers
'Visibility mode'



Used to control access to members of the class

public: can be accessed by functions outside of the class

private: can only be called by or accessed by functions that are members of the class

Protected: can't accessed from outside the class such as private member but they can be accessed by any **subclass** of this class. **(discussed later).**

Notes:-

- ✓ Can be listed in any order in a class
- ✓ Can appear multiple times in a class
- ✓ If **not specified**, the default is **private**

- ❑ As a rule of thumb, data members **should** be declared **private**
- ❑ Member functions **should** be declared **public**, Except member functions that are accessed **only** by other member functions of the class.
- ❑ Often useful to have **get** and **set** functions To access private members in controlled ways

Phase 2 :-

Implementation Phase

“Write your Code”

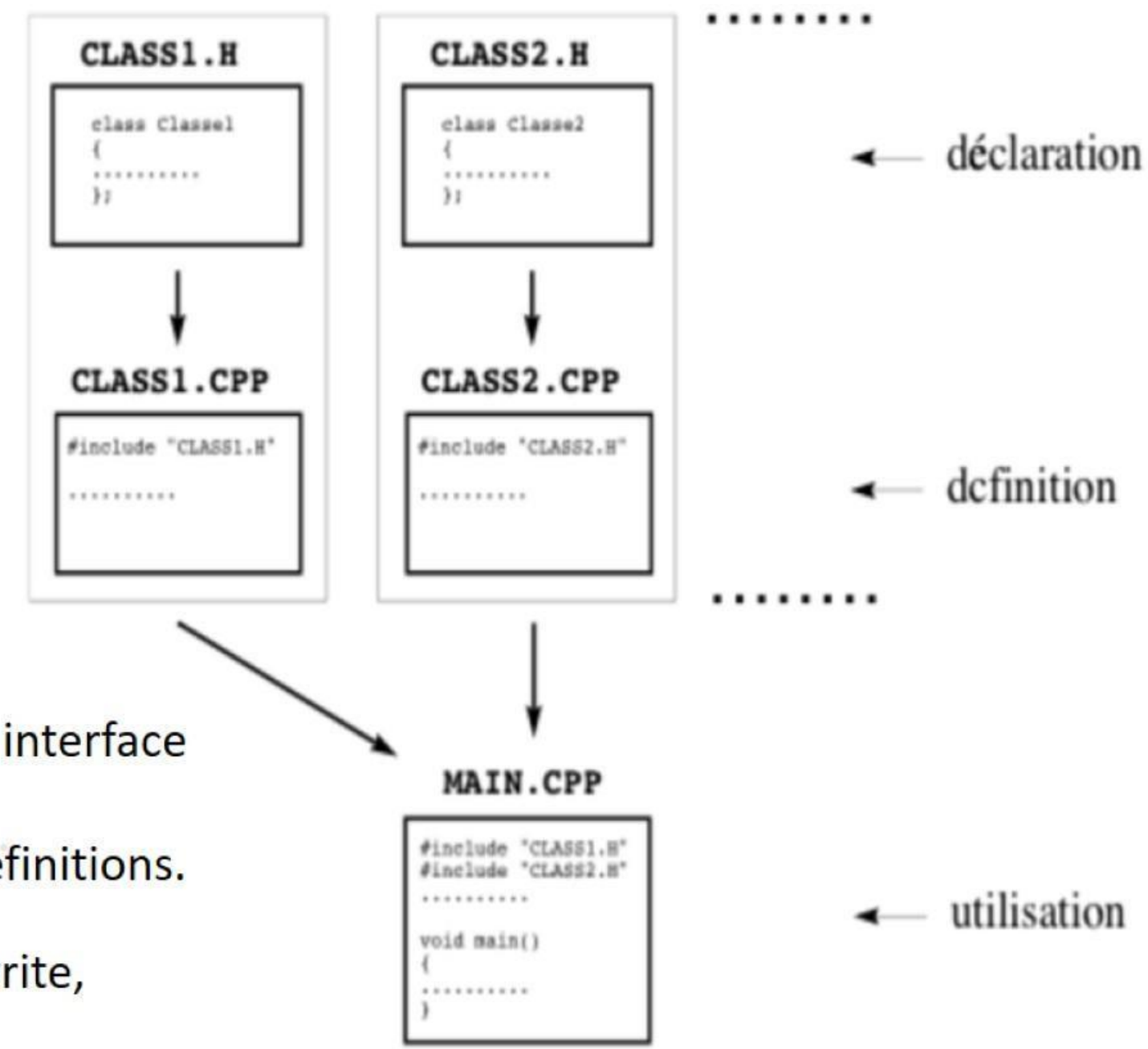
In C++, when writing classes you must understand separation of:

- ✓ **interface**: Declarations of functions, classes, members, etc.
- ✓ **implementation**: Definitions of how the above are implemented.

C++ implements this separation using two kinds of code files:

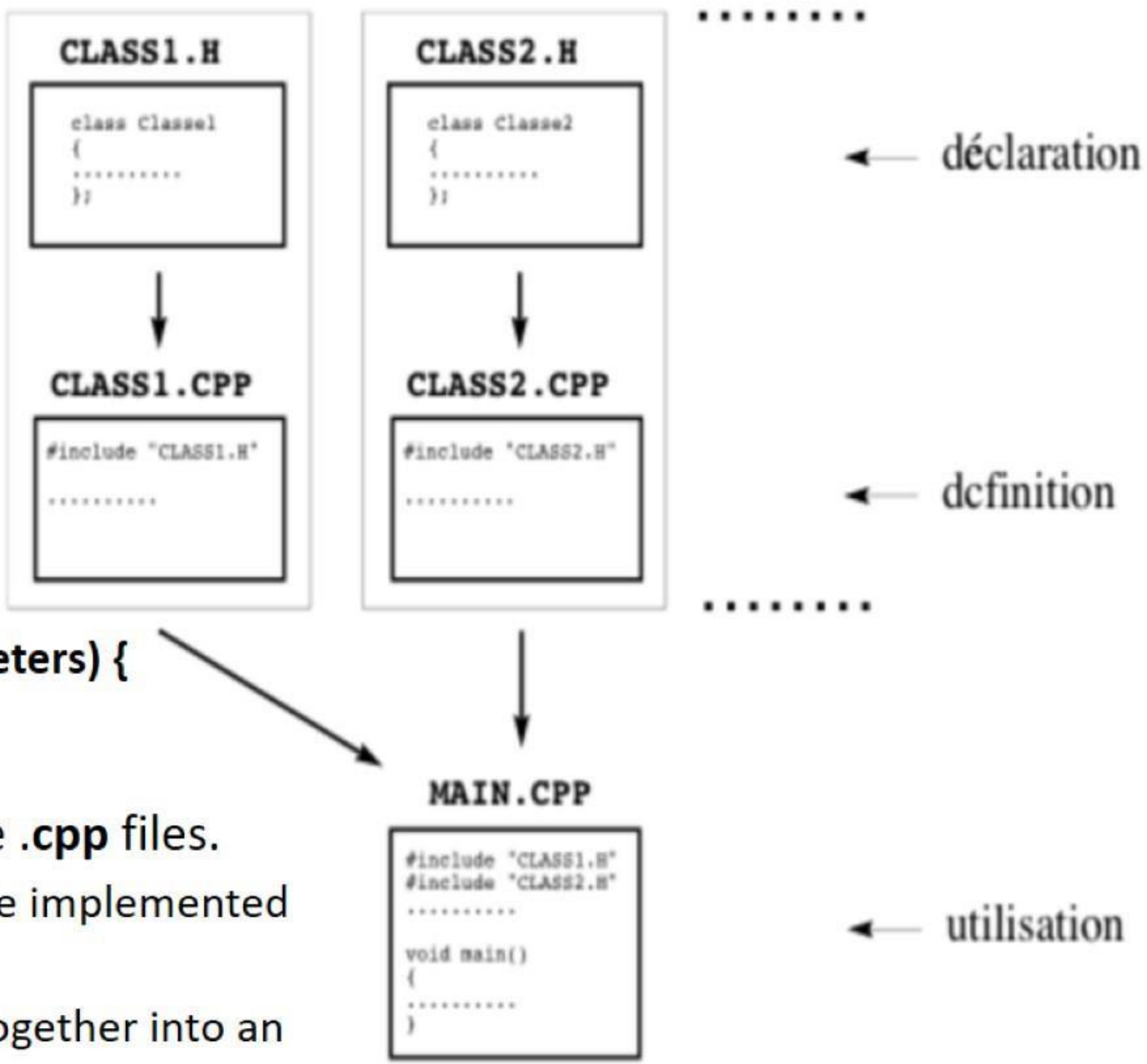
- ✓ **.h file**: A "**header**" file containing only interface (declarations).
- ✓ **.cpp file** : A "**source**" file containing definitions.

When you define a new class Foo, you write, **Foo.h** and **Foo.cpp**.



The content of **.h** files are a functions without any definition, Just a declaration "**Prototype**".

The body of these function will written inside the **.cpp** file using an operator called **Scope Resolution operator** (**::**)




```
returnType ClassName::methodName(parameters) {
   statements;
}
```

The content of **.h** files is "**#included**" inside **.cpp** files.

- Makes them aware of declarations of code implemented elsewhere.
- At compilation, all definitions are *linked* together into an executable.

Empolyee.h*

(Global Scope)

```cpp
#ifndef _classname_h
#define _classname_h
#include<iostream>
using namespace std;

class Employee{
    //data member
private:
    int empid;
    string empname;
    float salary;

    //function member as prototype
public:
    // just declration without defination
    void getdata();
    void Displayinfo();
};
#endif
```

Employee.cpp* Empolyee.h*

(Global Scope)

```cpp
#include"Empolyee.h";

/*
there are 2 method
need to implemented, we will use
an operater called Scope Resolution
to implement them outside the class
*/

void Employee::
```

Displayinfo
empid
empname
getdata
salary

public : void Employee::Displayinfo()
File: Empolyee.h

```cpp
void Employee::getdata(){
    empid=100;
    empname="ABC";
    salary=10000.0;
}
void Employee::Displayinfo(){
    cout<<"Employee Id : "<<empid<<endl;
    cout<<"Employee Name : "<<empname<<endl;
    cout<<"Employee Salary : "<<salary<<endl;
}
```

Constructor

Constructor

- It is public function member
- It is automatically called when a new object is created (instantiated).
- It initializes data members.
- It has same name as class.
- It is a special member function that has no return type

Constructor

- Several constructors acts like function overloading
- If you do not specify a constructor, the compiler generates a default constructor for you.
- The default constructor is one that expects no parameters and has an empty body.

```
Circle () {}
```

→ **default constructor**

- A simple instantiation of the class (with no arguments) calls the default constructor.

```
Circle c;
```

Constructor

- You can have more than one constructor in a class, as long as each has a different list of arguments.
- To create a constructor that takes arguments:

```
Circle(double r)
{
   radius = r;
}
```

- You can pass arguments to parameters when creating an object:

```
Circle c(7);
```

Constructor

Circle
- radius: double
+ Circle()
+ Circle(double)
+ getArea(): double
+ getCircumference: double

Instances

c1:Circle
- radius=1
+ getArea(): double
+ getCircumference(): double

c2:Circle
- radius=5
+ getArea(): double
+ getCircumference(): double

```cpp
#include <iostream>
using namespace std;
class Circle
{
  private:
    double radius;
  public:
    Circle() {
       radius = 1.0;
    }
    Circle(double r)
    {
       radius = r;
    }
    double getArea()
    {
      return radius * radius * 3.14;
    }
    double getCircumference()
    {
      return 2 * radius  * 3.14;
    }
};
```

```cpp
int main()
{
  Circle c1; //invoke default constructor
  Circle c2(5); //invoke overload constructor
  //c1.raduis = 5; //error
  cout<<"Area1: "<<c1.getArea()<<endl;
  cout<<"Area2: "<<c2.getArea()<<endl;
}
```

```
Area1: 3.14
Area2: 78.5
```

Circle objects in memory

Constructor Types:

1. Default constructor “constructor without any parameter”

```
class Counter
{
private:
    unsigned int count; //count
public:
    Counter():count(0)  //constructor
    { count++; }
    int get_count() //return count
    { return count; }
};
int main(){
    Counter c1, c2; //define and initialize
    cout << "\nc1=" << c1.get_count(); //display again
    cout << "\nc2=" << c2.get_count() <<endl;

    system("Pause");
}
```

Constructor Types:

2. Parameterized constructor "constructor with parameters" used for initialize data member

```
class Distance
{
private:
    int feet;
    float inches;
public:
    Distance() : feet(0), inches(0.0)  //constructor (no args)
    { }
    //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    { }
};

int main(){
    Distance dist1, dist3; //define two lengths
    Distance dist2(11, 6.25); //define and initialize dist2

    system("Pause");
}
```

Constructor Types:

3. Copy constructor "used when 1 object of class initialize another object of the same class"

```
class Distance
{
private:
    int feet;
    float inches;
public:
    Distance() : feet(0), inches(0.0)  //constructor (no args)
    { }
    //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    { }
};

int main(){
    Distance dist1(11, 6.25); //two-arg constructor
    Distance dist2(dist1); //one-arg constructor
    Distance dist3 = dist1; //also one-arg constructor

    system("Pause");
}
```

Default constructors

- A constructor whose parameters all have a default argument is considered a default constructor.
- It would be an error to create a constructor that accepts no parameters along with another constructor that has default arguments for all its parameters. compiler don't able to resolve which constructor to execute.

```
Circle(){}//default constructor defined by compiler
```

```
Circle(){//default constructor with default argument
  radius=1;
  color="red";
}
```

Error: 'Circle()' cannot be overloaded with Circle().

Classes with no default constructors

- If the class has an overloaded constructor, then the compiler doesn't provide the default constructor.
- It is the responsibility of the programmer to provide the default one.

```
#include <iostream>
using namespace std;
class Circle
{
  private:
    double radius;
  public:
    Circle(double r)
    {
       radius = r;
    }
    double getArea()
    {
      return radius * radius * 3.14;
    }
    double getCircumference()
    {
      return 2 * radius  * 3.14;
    }
};
```

```
int main()
{
  Circle c1; //ERROR: cant find Circle() constructor
  Circle c2(5); //invoke overload constructor
  cout<<"Area1: "<<c1.getArea()<<endl;
  cout<<"Area2: "<<c2.getArea()<<endl;
}
```

Error???

If the class has an overloaded constructor, then the compiler doesn't provide the default constructor.